Module Code	CS351
Module Name	Programming 3
Module Lead	Dr Warren Chan (warren.chan@strath.ac.uk)
Assessment Type	Class Test
Assessment Name	Group Project
Assessment Weighting	45% of module assessment
Limitations (e.g. Time, Word Count)	This project must be implemented in Java using IntelliJ You must use some form of version control and provide a link to share your repository
Deadline for submission and Timings (You should ensure that you familiarise yourself with the penalties for late submission**)**	5pm on Monday 30th November 2026
Arrangements for submission	This is a group submission assessment. Only one student per group needs to submit via the link provided in MyPlace titled "Group Project". The repository link should be shared with warren.chan@strath.ac.uk
Return of Marks and Feedback	Marks and feedback will be provided within 15 working days.
Learning Outcomes Covered by this Assessment	L2: Appreciate the problems of process and thread synchronisation and understand concurrency mechanisms. L3: Appreciate the issues that affect the design, implementation and performance of multiprocessor systems and to understand current approaches to ameliorate these. L4: Demonstrate an understanding of OS protection and security mechanisms. L5: Display practical skills necessary to develop thread-safe concurrent applications.
Use of Generative AI/LLM's (e.g. ChatGPT)	You are NOT permitted to use any AI tool to assist in the preparation of answers in this assessment.

CS351 Programming 3

Group Project

Available Marks: 100
Weighting: 45% of module total

Submission deadline: 17:00 Monday, 30th November, Week 10

Port Wander Concurrent Spaceport Management System

Project Overview

As a group, you must design, implement and test a concurrent spaceport management system in Java.

Port Wander is a large orbital facility that manages space transports arriving, receiving maintenance, loading cargo and departing.

Multiple control terminals connect to a central server. The server is responsible for managing transports, docking bays, maintenance teams and cargo.

The rules provided in this specification define how the spaceport operates.

Project Specification

Space Transports

Each transport has:

- A unique identifier.
- A current state.
- An allocated docking bay, if applicable.
- An allocated maintenance team, if applicable.
- A maximum cargo capacity.
- A current cargo quantity.
- A scheduled departure time.

Transports progress through the following states:

WAITING → DOCKED → SERVICING → READY → DEPARTED

Only valid state transitions are permitted.

Docking Bays

Each docking bay may be AVAILABLE, OCCUPIED or CLOSED.

A waiting transport may be assigned to an available docking bay. Each bay can accommodate one transport at a time.

The system must support **exchanging the docking bays of two transports**, provided both transports are in either the DOCKED or READY state.

An exchange must either complete successfully for both transports or leave their original allocations unchanged.

The administrator may close or reopen unoccupied docking bays.

Maintenance Teams

Maintenance teams may be AVAILABLE or BUSY.

An available team may be assigned to a DOCKED transport, changing the transport's state to SERVICING.

Completing maintenance changes the transport to READY and releases its maintenance team.

A maintenance team cannot service multiple transports simultaneously.

Cargo Management

Each transport has a maximum cargo capacity and a current cargo quantity, measured in cargo units.

Control terminals must support the following operations:

- **Load cargo:** Add cargo units to a transport.
- **Unload cargo:** Remove cargo units from a transport.
- **Transfer cargo:** Move cargo units between any two transports.

Cargo operations are permitted when a transport is in the DOCKED, SERVICING or READY state.

The following restrictions apply:

- Cargo quantities must never exceed a transport's capacity or fall below zero.
- Transfers require the source transport to contain sufficient cargo and the destination to have sufficient available capacity.
- A transfer must either complete successfully for both transports or leave both cargo quantities unchanged.

Multiple control terminals may perform cargo operations simultaneously.

All successful cargo operations must be recorded in the event log.

Scheduled Departures

The server must operate a background thread that periodically checks transport departure times.

A transport may depart when it is READY and its scheduled departure time has been reached.

Departure changes its state to DEPARTED and releases its docking bay.

Background departures may occur while control terminals are performing other operations.

Clients and Administration

Multiple control terminals must be supported simultaneously. A command-line interface is sufficient.

Clients must be able to:

- View transports, docking bays, maintenance teams and cargo quantities.
- Allocate and exchange docking bays.
- Assign maintenance teams and complete maintenance.
- Load, unload and transfer cargo.

- View the event log.

The administrator must be able to:

- View all transports and resources.
- Add new transports.
- Close or reopen docking bays.
- View connected clients and the event log.

The server must validate all requested operations.

Events and Persistence

Successful state-changing operations must be recorded in a single shared event log, including a timestamp and relevant transport/resource information.

Connected terminals must receive notifications of significant events, including those generated by other terminals and background processing. Notifications must not prevent a client from accepting operator commands.

Transport, docking bay, maintenance team, cargo and event-log information must be persistent. Data may be saved when the server shuts down cleanly.

Sample data will be provided for transports, docking bays and maintenance teams.

System Constraints

The system must observe the following constraints:

- A docking bay cannot contain more than one transport.
- A transport cannot occupy more than one docking bay.
- A maintenance team cannot be allocated to multiple transports.
- Associated transports, docking bays and maintenance teams must record consistent information.
- Cargo quantities must remain between zero and each transport's maximum capacity.
- Cargo transfers must not result in cargo being lost or duplicated.
- Transports must follow valid state transitions.
- Failed operations must not partially modify system state.
- Each successful state-changing operation must create the appropriate event-log entry exactly once.

Concurrency and Thread Safety

In addition to the functionality specified above, your system must successfully handle concurrency and thread-safety risks. The following are possible examples:

- Two or more terminals attempting to modify the cargo quantity of the same transport at the same time.
- Two or more terminals attempting to allocate the same docking bay.
- Two or more terminals attempting to allocate maintenance teams to the same transport.
- Multiple docking bay exchanges taking place at the same time.
- An administrator attempting to close a docking bay while a terminal attempts to allocate it.
- A scheduled departure occurring while a terminal is performing an operation involving the same transport.
- Multiple operations updating the event log while other terminals are viewing it.

Your system must maintain the specified constraints during concurrent operations. You are expected to identify and test relevant concurrency risks, including those not explicitly listed above.

Threading and Performance

The server must support multiple clients using threads. For full marks, a thread pool should be used to manage client connections.

You must make meaningful use of the Java synchronized keyword. Other appropriate concurrency mechanisms may also be used.

Your system should make effective use of concurrency. Independent operations should be able to progress concurrently without unnecessary blocking.

Your design should consider synchronisation, deadlocking, inter-process communication and performance.

Group Responsibilities

Groups will normally consist of four members. Each member should have a primary technical responsibility.

Responsibility	Typical Areas
Client & Communication	Client application, protocol, networking, notifications
Transport & Cargo Operations	Transport states, cargo operations, transfers, validation
Resource Management & Server Services	Docking bays, maintenance teams, departures, administration, persistence
Integration & Verification	System integration, automated testing, concurrency testing

These identify primary responsibility only. They are not exclusive areas of work, and all members remain responsible for producing an integrated system.

Testing is a shared responsibility and a core component of this project. Concurrency issues are difficult to diagnose and require deliberate, structured testing strategies; they cannot be successfully addressed as an afterthought.

For three-person groups, responsibilities may be combined.

Responsibilities may change during development where necessary. Significant changes should be recorded in the contribution diary.

The group should establish a basic working client/server system early so that completion of the project does not depend entirely on one member delivering an isolated subsystem.

Source Control

You must use GitHub, GitLab or another agreed source-control system.

Every member must make identifiable technical contributions. If none are evident, a mark of zero may be awarded to that individual.

The teaching team must be given access to the repository before submission.

Contribution Diary and Peer Assessment

The contribution diary must identify each member's initial primary responsibility and record weekly design, implementation, testing and integration work. Significant changes in responsibility should also be recorded.

Following submission, each student will allocate peer-assessment points between all group members and provide commentary justifying the allocation.

The contribution diary and source-control history may be used to support or challenge peer assessments where substantial differences in contribution are reported.

You assess your peers through allocating point to indicate the value contributions to the project. Your allocation of points should total 100 (or less to avoid fractional parts) and includes assigning a value to yourself. For example, if you have a group of 4 and you feel everyone in the team contributed equally to the project you would allocate as follows:

Member 1: 25

Member 2: 25

Member 3: 25

Member 4: 25

For a group of 3 with equal workload this would be:

Member 1: 33

Member 2: 33

Member 3: 33

You should also add commentary to justify your point allocation. Note if one team member has been allocated double the points of another (e.g. 50 to 25) then this suggests double the amount of work has been undertaken. This must be evident through your comments and any documentation supplied previously. If the marks allocated do no not appear to be justified or are vastly out of line with documentation they may be disregarded.

Members earning the highest mark are awarded the full group mark, while lower scoring member receive a percentage of the group mark, proportional to the difference of the highest scoring members.

Marking Scheme

Design – 20 Marks

Include appropriate design documentation such as UML, sequence diagrams, state diagrams or CRC cards.

Your design should demonstrate understanding of:

- Threading and shared resources.
- Synchronisation and deadlocking.
- Client/server communication.
- Interactions between transports, docking bays, maintenance teams and cargo.
- Performance implications of concurrency decisions.

Explain and justify significant design decisions and identify important limitations.

Criterion	Marks
Threading and concurrency	8
Server/client functionality and communication	8
Clarity and justification	4
Total	**20**

Implementation – 50 Marks

Implement the specified system with appropriate input validation, error handling and exception handling.

Criterion	Marks
Server functionality	9
Client functionality	8
Client threading and notifications	6
Server threading and background processing	8
Inter-process communication	7
Thread safety and concurrency performance	12
Total	**50**

Of the 12 marks for thread safety and concurrency performance, 9 marks assess correctness under concurrent operations and 3 marks assess effective concurrency and avoidance of unnecessary blocking.

Testing – 25 Marks

Your submission must include automated testing, manual testing and appropriate testing documentation.

Automated testing should use JUnit or an equivalent automated testing approach.

You must identify relevant concurrency risks and develop automated tests capable of exposing them. These should include tests involving:

- Cargo operations.
- Docking bay and maintenance team operations.
- Background processing interacting with client operations.

Concurrency tests must involve operations executing concurrently and automatically verify the results. Sequential tests alone are not sufficient evidence of thread safety.

Your documentation should explain what is being tested, the expected results, actual results and why the tests are appropriate.

Criterion	Marks
Automated Testing (subtotal)	16
Functional testing	3
Concurrent cargo operations	5
Concurrent docking and maintenance operations	4
Background processing and client interactions	4
Manual Testing	4
Documentation and Justification	5
Total	**25**

Video Demonstration – 5 Marks

Submit a video of no more than 6 minutes demonstrating the principal functionality of the completed system.

You do not need to explain the design or testing documentation in the video.

Submission

You should submit the following as a group no later than 17:00 on Monday 30th November 2026:

- Project source files and required data files.
- Design documentation.
- Testing documentation.
- Contribution diary.
- Repository link.
- Video demonstration.

Only one member needs to submit, but every member of the group should review what has been submitted.